Strings

~ G. Anushiya Rachel
Assistant Professor, CSE
Shiv Nadar University Chennai

Assigning a String

- To assign a string to a variable, use single or double quotes:

$$a = "Hello"$$
$$b = 'World'$$

- To assign multi-line strings, using tripe quotes:

$$m = '''Hello$$
$$Good\ day'''$$

- $print(m)$ will result in

$$Hello$$
$$Good\ day$$

Indexing and Slicing

- To access characters in a sting, use indices.

$$s = 'python'$$
$$print(s[0], s[-2])$$

  - $s[0]$ would refer to $'p'$ and $s[-2]$ would refer to $'o'$
- Strings are immutable, therefore, equating $s[i]$ to another character is not supported. However, s can be equated to a different string altogether.
  - $s[2] = 'r'$ # Not supported
  - $s = 'new'$ # Supported
- Slicing – used to obtain a substring

$$s =' programming'$$
$$print(s[3:8])$$

  - This will result in the string, $'gramm'$

Slicing

- Let $s = 'python'$
- $s[:]$ – Entire string
  - $'python'$
- $s[1:]$ – Elements from index 1 to end
  - $'ython'$
- $s[:3]$ – Elements from index 0 to 2
  - $'pyt'$
- $s[:5:2]$ – Elements from 0 to 6 in steps of 2 (alternate elements)
  - $'pto'$
- $s[::-1]$ – Elements from the end to the beginning – Reverse of the string
  - $'nohtyp'$

String Operations

- String concatenation (+):
  - $s1 = 'abc'$
  - $s2 = 'de'$
  - $print(s1 + s2)$ # Prints $'abcde'$

- String multiplication by an integer (∗):
  - $print('ab' * 3)$ # Prints $'ababab'$

- String comparison using relational operators:
  - $s1 == s2$ # $False$
  - $s1 < s2$ # $True$

String Operations

- Length of a string using len:
  - $len(s1)$ # 3

- String membership test using in:
  - $print('a' in 'abc')$ # $True$
  - $print('f' in 'abc')$ # $False$

- Iterating through a string:
  - $for\ i\ in\ 'abc':$

    $print(i)$ # Prints each letter in the string

Some String Methods

- $upper(\quad), lower(\quad)$
  - $'abc'.upper()$ # Results in $'ABC'$
- $isalpha(\quad), isnumeric(\quad), isdecimal(\quad)$
- $replace$(string_to_be_replaced, replacement_string)
  - $'abc'.replace('a','d')$ # Results in $'dbc'$
- $lstrip(\quad), rstrip(\quad)$ - Removes leading and trailing spaces. To remove some other leading/trailing character, mention it within brackets ($'abc'.lstrip('a')$ results in $'bc'$)
- $split(\quad)$ - Splits the string at the spaces and returns a list of substrings ($'abc\ def'.split(\quad)$ results in $['abc','def']$)

Note: Use $\boldsymbol{help(str)}$ for the complete list of string methods.

Some String Methods

- $count(\quad)$ - Count the number of times a given substring appears in a string ($'hello'.count('el')$ results in 1.
- $join(\quad)$ - To concatenate strings separated by the given string
  - Example: $' '.join(['cd','ef','121'])$
    - This will result in $'cd\ ef\ 121'$
  - Example: $'a'.join(['cd','ef','121'])$
    - This will result in $'cdaefa121'$

String Interpolation

- Substituting values of variables into place holders in a string
- f-strings:
  - $s1, s2 = 'hello', 'world'$
  - $print(f'This\ is\ s1\ -\{s1\}\ and\ this\ is\ s2\ -\{s2\}')$
- % formatting:
  - $c = 1 + 2$
  - $s = 'python'$
  - $print('Printing\ variables\ in\ \%s: c = \%d + \%d = \%d'\ \%(s, 1, 2, c))$
  - This will print $'Printing\ variables\ in\ python: c = 1 + 2 = 3'$
- $format$ method
  - $'hello\ \{\}'.format(s)$ # Results in $'hello\ python'$

Regular Expressions (RegEx)

- Sequence of characters that defines a search pattern
- Module – re
- Find a pattern:
  - $re.findall(pattern, string)$
  - $re.search(pattern, string)$ – returns a match object
    - $match.group(\quad), match.start(\quad), match.end(\quad), match.span(\quad)$
  - $re.match(pattern, string)$ – finds a match only at the beginning of string and returns a match object
- Split the string at the pattern:
  - $re.split(pattern, string)$
- Replace
  - $re.sub(pattern, replacement, string), resubn$ – returns count too

RegEx Patterns

- [ ] ~ Used to specify a set of characters to be matched
  - Examples: $[a - e]$ or $[abcde]$, $[1 - 39]$ or $[1239]$
  - Character set can be complemented using ^.
    - Example: $[\hat{}a - c]$ – not a, b, c
- ^ is also used to check if a string starts with the specified character(s).
  - Example: $\text{re.find}all('\hat{}a', ' anckhad')$
- $ ~ used to check if a string ends with the specified character(s)
- $*$ ~ 0 or more occurrences
- $+$ ~ 1 or more occurrences
- ? – 0 or 1 occurrence
- . – any character

RegEx Patterns

- $\{n\}$ – n repetitions of the pattern to the left
- $\{n, m\}$ – at least n and at most m repetitions of the pattern to its left
- | ~ or
- ( ) – group
  - Example: $(a|b|c)ef$ – a or b or c followed by ef
- \ ~ to escape characters

Special Sequences

Character	Description	Example
\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"
\b	Returns a match where the specified characters are at the beginning or at the end of a word	r"\bain" r"ain\b"
\B	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word	r"\Bain" r"ain\B"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white space character	"\S"
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Programming Exercises

- Write Python programs for the following:
  1. Check if the given string is a valid name (That is, the string should only contain alphabets and possibly a '.' and spaces)
  2. Count the number of uppercase letters, lowercase letters, digits, and special characters in the given string.
  3. Reverse the given string and check if it is a palindrome.
  4. Check if a given substring is present in a given string.
  5. Remove the ith character in a string.
  6. Print the characters in a string, after removing duplicates, and find the frequency of occurrence of each character.
  7. Check if a given string contains only alphabets and find the number of vowels and consonants in the string.

Programming Exercises

8. Get a string from the user and replace a given character by another given character.
9. Find the characters in a string that appear an odd number of times.
10. Find the number of words in a given string.
11. Reverse each word in a given string.
12. Sort the characters of a given string (of alphabets) in alphabetical order.
13. Sort the characters of a string based on their frequency of occurrence.
14. Obtain a string and a rotated version of a string. Find the number of times the rotated version of the string must be rotated to the right and to the left to obtain the actual string.
15. Check if a given string can be made empty by recursively deleting a given substring.

Programming Exercises

16. Get a sentence from the user and replace the first letter of each word by the next letter in the alphabet sequence (E.g.: this is a zebra. → uhis js b aebra.) Hint: Use the functions *ord* and *chr*
17. Find the smallest substring that contains all the characters of the given string.
18. Count the number of substrings with the same first and last characters in a given string.
19. Break a given string into multiple lines with N characters each. Get the value of N from the user.
20. Implement the Caesar's cipher.

Programming Exercises

21. Check if a given string is a valid email id. An email id usually contains a recipient name, @, domain name, and top-level domain name (.com, .org, .co.in, etc.). Assume that special characters allowed in the recipient name are .,-, and _ (dot, hyphen, and underscore) and those allowed in the domain name are hyphen (-) and underscore (_). Also, the top-domain name must have at least 2 characters.